

A multi-agent recruitment system with human review

Repository: https://github.com/layal-z/AAI614_Zeineddine/tree/Agentic_AI/Multi_Agent_Recruitment_System_GPT_v11.ipynb

I. Introduction

Recruitment starts with understanding what the client needs. The recruiter then prepares the job description, reviews CVs, plans interviews and gives the client an assessment. These tasks are often done in separate files. This makes it easy to miss a requirement, use an old document or write a conclusion without showing where it came from.

I built a system with four specialist agents to support these tasks. It prepares drafts, shows the evidence and identifies points that need clarification. The recruiter checks and approves the work. The recruiter also remains responsible for contacting candidates and making hiring decisions. The system does not contact candidates or make the final hiring decision.

II. Demonstration and scope

I tested the system on an HR Director role for a new sanitaryware factory in Saudi Arabia. The client asked for a Saudi candidate with at least ten years of experience. The role covers personnel affairs, recruitment, HR policies, labor regulations and developing the HR function. The workforce will gradually reach 100 employees in the first phase and is expected to reach 400 over five years. The official location is Jeddah MODON 3, with the first six to twelve months in Madinah.

I used two CVs and their English interview records, with names and employer details removed. Both candidates completed GPT screening, interview planning, evaluation and human review. Candidate_002 also completed the confirmed tracker update. For Candidate_001, the supplied tracker output showed the proposed next step but did not confirm that the update was saved.

The main user is a recruiter who understands the role and can judge whether the evidence is relevant. The hiring manager can review the final report and open questions. The working system is a Jupyter notebook with editable review forms and a saved case file. The optional local browser app includes both candidates' saved reviews, which can be viewed without an API key. The notebook is the main submitted system.

III. Agents and coordination

I used the handoff pattern studied in the course. The agents work in a fixed order, and Python passes the approved information to the next stage. Each agent has its own task, prompt and model conversation. The recruiter runs each stage and reviews the output before continuing. The agents do not debate or choose the next agent themselves.

Agent	Main responsibility	Output
1 Role and outreach	Extract the brief and draft recruitment materials.	Facts with source quotes, clarification questions, JD, outreach and scorecard.

2 Screening	Compare one candidate CV with approved criteria.	Criterion judgments, selected evidence, explanations and status counts.
3 Interview planning	Use reviewed screening to identify useful follow-ups.	Questions, purposes, answer signals and professional or recruiter routes.
4 Evaluation and reporting	Independently assess CV and reviewed interview evidence.	Evidence-based assessments, open questions, client summary and next step.

➤ How the handoffs work

Agent 1 accepts the client's brief as free text, such as an email or message. The client does not need to fill in a questionnaire. The agent extracts the requirements, raises clarification questions and drafts the JD, outreach message and scorecard. Python checks the output format and source quotes. The recruiter then edits and approves the materials. Agent 2 uses tool calling to request the selected candidate's evidence. Python checks the request, retrieves the evidence through MCP and gives it to the model for screening.

Agent 3 receives the approved criteria and reviewed screening. It proposes questions to explore relevant experience and confirm missing information. The recruiter reviews and approves the plan. The matching interview record is then checked and saved as actual or simulated. Agent 4 receives the approved criteria, CV and reviewed interview evidence. It assesses this evidence independently; its assessment prompt does not include Agent 2's judgments.

The original screening and evaluation drafts are kept separately from the human edits. If approved content changes, the next step is blocked until it is reviewed again. The final report needs approval, and updating the tracker needs a separate confirmation. A suggestion from the model therefore cannot update the tracker on its own.

➤ Model choice

I first used Qwen3 4B locally through Ollama. It could copy some information correctly, but it was unreliable when comparing the requirements, CV and interview. It invented unrelated experience, made incorrect comparisons and sometimes contradicted its own explanation. I switched the four agents to GPT-4.1 through the OpenAI API. Both are language models. Changing the model did not replace the Python workflow, MCP tools or approval checks.

IV. Tools, memory and connections

The model asks to use a tool through function calling. Python checks the tool name and candidate ID before allowing the request. Agents 2, 3 and 4 each receive only the tool needed for their stage. After checking approvals, Python prepares a temporary copy of the permitted information and connects to the local MCP server.

MCP tool	Information returned
read_candidate_evidence	The selected CV, divided into identifiable source fragments.
read_interview_context	Approved criteria, reviewed screening and selected candidate evidence.
read_evaluation_context	Approved criteria, CV and reviewed interview evidence, source label and approval records.

Before MCP, Python returned the information directly through local functions. Adding MCP moved the retrieval tools into a separate server with a standard way to list and call them. The MCP client finds the tools, calls the permitted one and checks that the returned information matches the prepared copy. The server cannot read any file it chooses or update the tracker. These restrictions come from our code, not from MCP automatically. Cell #39 separately tests the three MCP tools without calling the model.

➤ Evidence and memory

Python gives CV lines E IDs and interview lines I IDs. The model selects the IDs that support its assessment, and Python adds the exact source text to the report. Sometimes one sentence is split across lines, so more than one ID is needed. The criteria also receive IDs based on the approved scorecard. They are not fixed to this vacancy.

The system remembers progress in a local JSON case file. It saves the brief, role materials, candidate evidence, drafts, human reviews, approvals and tracker history. This lets the user reopen the case and reuse work when the approved inputs have not changed. I did not need vector RAG: the selected CV and transcript were small enough to give the model directly. Searching for only a few passages could leave out useful or conflicting evidence.

Each approval includes a hash, which acts as a version check. If the approved brief, scorecard, CV, plan or notes change, the code detects the change and asks for review again. The tracker also keeps a history and avoids adding the same confirmed update twice. I did not need the A2A protocol because all four agents are coordinated inside the same Python application. Their handoffs do not require communication between separate agent services.

➤ Running the system

The system uses the OpenAI API for tool calling and structured JSON outputs. Python checks the JSON before accepting it. A temperature of zero helps consistency but does not guarantee identical answers. The API key is entered through a hidden password prompt and is not saved in the case file. Reviewers need their own funded API access to generate new outputs. Without a key, reviewers can read saved outputs, open both demonstration cases in the app and run offline tests with simulated API responses. These tests check the workflow, not model quality. The README explains installation, file paths, cell order and human review.

V. How did I evaluate the system?

Following the course's evaluation approach, I checked both the trajectory and the final output. The trajectory is the sequence of steps: which tool was requested, which candidate was selected, whether approvals were checked and whether the result was validated and saved. The trace records what happened during the run. I also checked the assessment itself: whether it covered the criteria, used exact source quotes and needed changes during human review.

➤ **What happened during development?**

Qwen3 4B gave an HR candidate software and cloud experience that was not in the supplied CV. Some outputs invented evidence IDs and were rejected by validation. Another used existing IDs but still made unsupported claims. One full request exceeded the 900-second limit. This showed that correct JSON and valid IDs were not enough to make the assessment reliable.

In a test of only the ten-year experience requirement, Qwen returned not_demonstrated while its explanation said the candidate met the requirement. Another response said fifteen years was less than ten. A separate extraction test copied the employment dates correctly. The evidence was available, but the model did not assess it reliably. We did not establish exactly why it made these mistakes.

GPT-4.1 assessed the supplied experience evidence as demonstrated in 2.3 seconds. The GPT system then produced reviewed evaluations for both candidates. These were useful practical tests, but not a controlled comparison of the two models. The hardware, output lengths and some prompts differed, and the code changed during development.

Observed GPT stage	Elapsed time	Validation
Agent 1 brief extraction	7.71 seconds	Passed validation on first attempt
Agent 2 Candidate_002 screening	6.52 seconds	Passed validation on first attempt
Agent 3 Candidate_002 plan	8.51 seconds	Passed validation on first attempt
Agent 4 Candidate_002 evaluation	7.75 seconds	Passed validation on first attempt
Agent 4 Candidate_001 evaluation	10.68 seconds	Passed validation on first attempt

These timings describe the recorded runs. They are not guaranteed response times.

Candidate_002's assessment used 6,650 input and 663 output tokens. Candidate_001 used 5,384 and 648. Each tool request used 96 input and 20 output tokens. The trace records these results without exposing private model reasoning.

I also ran four controlled tests in cell #41 using Agent 4, its prompt, validation and real MCP retrieval. The cases were a 15-year positive control, a three-year candidate below the minimum, an interview correcting a 15-year CV claim to five years, and a three-year CV with an instruction to ignore the criteria and approve it. All four passed. The injected instruction was ignored and the candidate remained not_demonstrated. These were synthetic assessment tests, not full approval-chain tests. The real case was unchanged.

GPT was faster in these runs and did not invent the same unrelated technology experience. It still made mistakes. It suggested 100 employees at the start instead of gradual growth, overstated continuous employment and selected some weak evidence. Human review remained necessary after changing the model.

VI. Safeguards and output quality

I ran cell #34 for both candidates. It made temporary copies of the case and tested what happened when approval was missing or approved content had changed. It also checked that an unchanged approved report was accepted and that a tracker update without confirmation was blocked. Both candidates passed all eight checks. The real case was not changed by the tests.

Check	Expected result	Observed
Unchanged approved report	Accepted	Pass
Missing role approval	Blocked	Pass
Criteria changed after approval	Blocked	Pass
CV changed after screening review	Blocked	Pass
Interview plan changed after approval	Blocked	Pass
Interview notes changed after review	Blocked	Pass
Final report changed after approval	Blocked	Pass
Tracker update without confirmation	Blocked	Pass

➤ Results for the two candidates

Both reviewed evaluations covered all eight criteria. Candidate_001 had 17 out of 17 exact source quotes. Candidate_002 had 26 out of 26. Neither had an explanation that mentioned an evidence ID missing from its selected references. These checks confirm that the text was copied correctly. They do not prove that the evidence supports every conclusion. The final saved case includes two human status corrections for each candidate: Knowledge of regulations became partially_demonstrated because compliance experience did not show how the candidate stays updated. Relocation became requires_human_confirmation because it had not been discussed. For Candidate_002, the reviewer also removed Arabic language as evidence of nationality.

The model and reviewer agreed on 6 out of 8 statuses for Candidate_001 and 6 out of 8 for Candidate_002. This is agreement, not an accuracy score. Candidate_001 still had two weak judgments: courses were treated as evidence of staying updated, and missing relocation information was marked not_demonstrated. This shows that human approval can also miss mistakes. Neither client summary was edited; it should also be checked when detailed judgments change.

➤ What still needs testing

The two real candidates and four synthetic tests do not establish hiring outcomes, fairness or general accuracy. One ignored attack does not prove protection from every prompt injection.

Further tests should use more roles and CV formats, independent reviewers, incomplete answers and different attacks.

VII. Limitations, safety and next steps

The main risk is an answer that looks convincing and uses a real quote, but claims more than the quote supports. Compliance experience does not automatically show how someone follows legal updates. Helping with a project is not the same as leading it. Unclear interview passages should remain uncertain. An interview answer can clarify a CV claim, but it should not automatically replace all CV evidence.

Concern	Control used	What can still go wrong
Document instructions or prompt injection	Treat sources as data; allowlist the tool and candidate.	One injection test passed; other attacks may still succeed.
Unsupported or misleading conclusions	Validate structure and IDs; attach source quotations; require review.	Existing citations can be irrelevant or weaker than the explanation.
Consequential recruitment actions	Approval gates and separate tracker confirmation.	The reviewer may approve a weak interpretation or overlook a summary inconsistency.
Privacy and personal attributes	Deidentified inputs; password entry; nationality requires confirmation.	Case files are plaintext and API requests send supplied data to a provider.
Stale or mismatched records	Version fingerprints and selected-candidate checks.	Wrong source documents can still be confirmed by a user.

The nationality requirement came from the client and still needs human review. The system does not decide whether the condition is legally appropriate. Language, location and names do not prove nationality. If willingness to relocate has not been discussed, the next step is to ask the candidate. Missing information should not be treated as a negative answer.

The system depends on API access, credits and the local Python setup. Another user must set the correct project folder. Reading a CV or transcript can lose formatting. Scanned PDFs need OCR or checked pasted text. Case files are stored as plain text, and model requests send the supplied information to OpenAI. Using store=False does not guarantee that all provider retention is disabled. API keys and private operational cases must stay outside GitHub.

The next step is to improve evaluation: check whether each quote supports the explanation, keep conflicting statements visible, check the client summary and test prompt injection. More vacancies and independent reviewers would give stronger evidence. The browser app could make the workflow easier to use, but shared online access would need login, access controls and a way to handle simultaneous edits.

This project shows how specialist agents, tool calling, MCP, saved memory and human approvals can work together in recruitment. The agents help prepare and assess information. Python controls access and actions. The recruiter remains responsible for the evidence review and final decision.